TicketFlow — Decisões do projeto

Oct 22, 2026 · @caua[

Visão geral

O TicketFlow faz a triagem automática de chamados de suporte: classifica cada ticket por categoria e prioridade, pede informação ao cliente quando o ticket está incompleto e aprende com as correções dos atendentes.

Problema de negócio: empresas recebem milhares de tickets por dia, e alguém precisa ler cada um para decidir quem resolve. Sem triagem, um cliente cobrado em dobro pode esperar dias atrás de uma dúvida simples.

Objetivo de portfólio: demonstrar os dois diferenciais pedidos em vagas de ML/IA:

- ML em produção, com monitoramento, versionamento e retreino;
- agentes de IA com orquestração de múltiplos agentes e uso de ferramentas.

O projeto aplica práticas de produção num cenário simulado; não tem a escala de uma empresa real.

Decisões tomadas

Nove decisões estão fechadas; cada uma tem o motivo que a sustenta numa entrevista.

Decisão	Escolha	Por quê
Nome	TicketFlow	Descreve o fluxo de triagem: o sistema encaminha; nos casos difíceis, quem decide é o analista
Tema do projeto	Triagem de chamados de suporte	Caso que toda empresa reconhece; correções dos atendentes dão um sinal natural de retreino; mostra NLP e diversifica o portfólio em relação ao S4Lake
Dataset	Tobi-Bueck/customer-support-tickets: 61.765 tickets em inglês e alemão, licença CC BY-NC 4.0	Tem fila, prioridade e resposta do atendente; o Bitext é sintético, repetitivo e já muito usado em portfólios

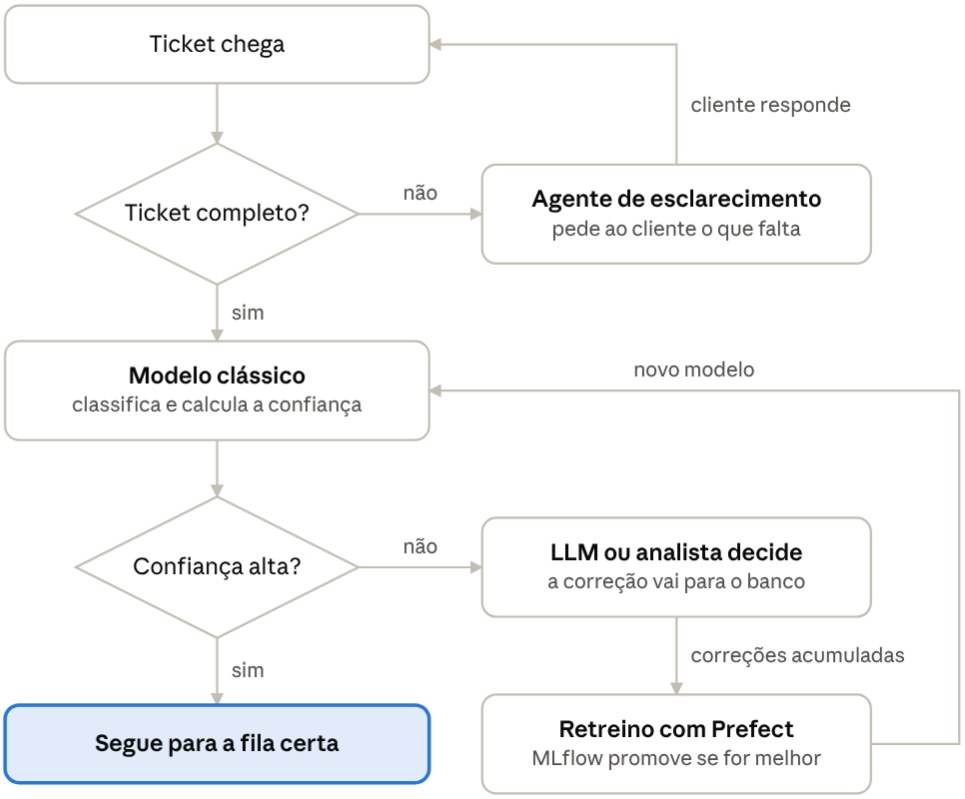
Decisão	Escolha	Por quê
Categorias	Filas (`queue`) do dataset	Os rótulos já existem nos dados; cada fila é um time que resolve
Quem classifica o ticket	Híbrido: modelo clássico primeiro, LLM só quando a confiança é baixa	Custo quase zero nos casos fáceis; confiança calculada e verificável; existe um modelo para retreinar
Prioridade	Modelo prevê baixa, média ou alta; regras de negócio elevam para P1	P1 é raro, e modelos erram nas classes raras; um caso crítico não pode depender de uma probabilidade
Ticket incompleto	Tratado à parte, por um agente de esclarecimento	Ticket incompleto pede informação ao cliente; modelo inseguro pede decisão de um analista
LLM	Via API	Mais qualidade que um modelo local; custo controlado porque o LLM só atua nos casos difíceis
Nuvem	AWS adiada; projeto roda local primeiro, com MLOps open source	Foco no sistema antes da infraestrutura; MLflow e Evidently funcionam em qualquer nuvem

Regra para criar agentes: um agente separado só existe quando tem responsabilidade, ferramentas ou permissões próprias. Exemplo: quem redige a resposta ao cliente não pode escalar chamados.

Fluxo da classificação híbrida

O ticket incompleto volta ao cliente; o completo passa pelo modelo clássico, e só o de baixa confiança vai para o LLM ou o analista.

Só o caso difícil gasta LLM ou tempo de analista



fluxo de classificação híbrida · 2 decisões, 1 ciclo de retreino

As correções dos analistas alimentam o retreino, que só promove o novo modelo se ele for melhor que o atual.

Ticket	Completo?	Modelo clássico	Resultado
"Fui cobrado duas vezes no cartão"	Sim	Financeiro, 96%	Segue direto para o financeiro
"O negócio travou, ajeitem logo"	Não	Não chega aqui	Agente pede sistema, print e passos ao cliente
"Após atualizar o app, fui cobrado de novo"	Sim	Financeiro, 55%	LLM ou analista decide; a correção vira dado de retreino

Prioridade e SLA

Os três níveis do dataset viram P2, P3 e P4; o P1 vem só das regras de negócio.

Nível	Nome	Critério	SLA alvo	Quem atribui
P1	Crítica / Urgente	Sistema fora do ar, interrupção total do negócio, vazamento de dados ou bloqueio que impede o cliente de trabalhar	Menos de 1 hora	Regras de negócio
P2	Alta	Operação impactada significativamente, perda de funcionalidade importante sem solução alternativa	4 a 8 horas	Modelo (alta)
P3	Média	Problema menor com solução alternativa disponível, ou dúvida de uso cotidiano	24 horas	Modelo (média)
P4	Baixa / Planejada	Pedido de novo recurso, melhoria estética, dúvida teórica ou manutenção agendada	48 a 72 horas	Modelo (baixa)

Agentes

Quatro agentes, orquestrados pelo LangGraph; cada um tem ferramentas e limites próprios.

Agente	Faz	Ferramentas	Não pode
Esclarecimento	Detecta ticket incompleto e pede ao cliente o que falta	Enviar mensagem ao cliente	Classificar ou resolver
Revisor	Analisa os casos de baixa confiança e sugere a categoria, com justificativa	Ler o ticket e as 2 categorias mais prováveis do modelo	Encaminhar sozinho: o analista confirma
Resolução (RAG)	Busca tickets parecidos já resolvidos e redige a resposta	Busca na base de conhecimento	Enviar sem aprovação humana

Agente	Faz	Ferramentas	Não pode
SLA	Acompanha o prazo de cada prioridade e escala o que vai estourar	Consultar a fila e notificar o responsável	Alterar a classificação

Qualidade do modelo e retreino

Toda decisão do modelo é medida, e um modelo novo só entra em produção se provar que é melhor.

- **Limiar de confiança:** escolhido na validação como o menor limiar cujo acerto nos tickets automáticos atinge a meta de negócio; assim a automação é máxima sem perder qualidade.
- **Calibração:** diagrama de confiabilidade e ECE (erro de calibração esperado); se o modelo estiver descalibrado, corrigir com `CalibratedClassifierCV`.
- **Gatilho de retreino:** drift detectado pelo Evidently, ou correções acumuladas acima de uma fração do conjunto de treino.
- **Portão champion/challenger:** o modelo novo só substitui o atual se ganhar num conjunto de teste fixo e nas correções recentes; se perder, fica registrado no MLflow e o atual continua.
- **Drift simulado:** numa segunda fase, tickets em português são injetados de propósito para mostrar o monitoramento detectando a mudança e o retreino resolvendo.
- **Métricas de negócio:** taxa de automação, acerto nos tickets automáticos, custo por ticket, calibração e prazos de SLA cumpridos.

Escala e desempenho

O projeto não tem a escala de uma empresa real, então mede seus limites com números em vez de supor.

- **Processamento em lote:** classificar os 61.765 tickets do dataset de uma vez e registrar o tempo total e o tempo por ticket.
- **Teste de carga da API:** simular muitos tickets chegando ao mesmo tempo com o Locust e medir quantos tickets por segundo a API aguenta e com que latência.
- **Gargalo:** identificar qual etapa limita o sistema (modelo clássico, chamadas ao LLM ou banco) e documentar o que mudaria para escalar.

Stack proposta

Tudo roda localmente com ferramentas open source; cada tecnologia cobre uma parte dos dois diferenciais.

Camada	Tecnologia	Por quê
Base	Python	scikit-learn, MLflow, Evidently, LangGraph e Langfuse têm bibliotecas nativas em Python, então todo o back-end fica numa linguagem só, sem camadas de integração
Base	Docker e docker-compose	Sobe o sistema inteiro com um comando, em qualquer máquina
Base	PostgreSQL	Guarda tickets, classificações e as correções dos atendentes, que alimentam o retreino
ML em produção	scikit-learn	Treina o classificador de categoria e prioridade, com probabilidade verificável
ML em produção	MLflow	Registra cada treino e versiona os modelos no Model Registry
ML em produção	Evidently	Detecta drift nos tickets que chegam; é o gatilho do retreino
ML em produção	Prefect	Orquestra o retreino: junta correções, treina, compara e só promove se o novo modelo for melhor
Agentes	LangGraph	Orquestra os agentes como grafo com estado; suporta aprovação humana no fluxo
Agentes	LLM via API ou Ollama local	API dá mais qualidade; Ollama tem custo zero (a decidir)
Agentes	ChromaDB e embeddings	Base de conhecimento para a busca por similaridade (RAG)
Agentes	Langfuse	Tracing dos agentes, versionamento de prompts e avaliações
Interface	FastAPI	Expõe o sistema como API para outros softwares da empresa

Camada	Tecnologia	Por quê
Interface	Next.js e TypeScript	Tela do atendente com a fila e as respostas dos agentes atualizando em tempo real; tipagem do TypeScript evita erros entre front e API. O atendente corrige a classificação ali, e a correção alimenta o retreino
Qualidade	pytest e GitHub Actions	Testes e avaliações a cada commit, para uma mudança de prompt não quebrar o sistema

Segurança

Nenhum token ou chave entra no repositório; o vazamento mais comum é uma chave publicada no GitHub, não uma senha descoberta.

- Tokens e senhas ficam num gerenciador de senhas (Bitwarden ou KeePassXC), nunca em bloco de notas.
- O código lê credenciais de variáveis de ambiente; o arquivo `.env` fica no `.gitignore`.
- Token que já apareceu em repositório, print ou mensagem é revogado e substituído.
- Na fase AWS: MFA no usuário root, usuário IAM com permissões mínimas, credenciais temporárias via IAM Identity Center e alerta no AWS Budgets.

Futuras implementações

A AWS é a primeira evolução, depois que o sistema estiver completo rodando local.

1. Deploy na AWS com S3, SQS, Lambda, EventBridge, CloudWatch e Bedrock, priorizando serviços cobrados por uso.
2. Infraestrutura como código com Terraform, para recriar o ambiente numa conta nova (o plano Free fecha a conta em 6 meses).
3. CI/CD com OIDC, para o GitHub Actions acessar a AWS sem chaves guardadas.
4. Shadow deployment: o modelo novo roda em paralelo ao atual até provar que é melhor.
5. Dashboard de monitoramento no Grafana: drift, custo por ticket e latência dos agentes.
6. Integração com ferramenta real de tickets (Jira Service Management ou Zendesk) via API.

Pendências

Estas decisões precisam ser fechadas antes ou durante a implementação.

- [ ] Provedor e modelo do LLM via API (priorizar um modelo pequeno e barato)
- [ ] Meta de acerto nos tickets automáticos, que define o limiar de confiança (por exemplo, 95%)
- [ ] Fração de correções acumuladas que dispara o retreino (por exemplo, 5% do treino)
- [ ] Regras que elevam um ticket para P1 (palavras-chave, filas ou tipo de incidente)
- [ ] Quais filas do dataset viram categorias, e se as filas pequenas serão agrupadas